Helion++: Advanced Predictive Smart Home Automation

Software Project Lab 3 Technical Report

Submitted by:
Dipshikha Das
BSSE-1218
Institute of Information Technology
University of Dhaka

Date of Submission:
1st December, 2024

Supervised by:

Dr. Md. Nurul Ahad Tawhid
Associate Professor
Institute of Information Technology
University of Dhaka

External Supervisor:

Dr. Adwait Nadkarni
Associate Professor
Department of Computer Science
William & Mary

Supervisor's Signature:



IIT
University of Dhaka

Letter of Transmittal

Date: December 1, 2024

BSSE 4th Year Exam Committee
Institute of Information Technology,
University of Dhaka

Subject: Submission of technical report for **Helion++** - Advanced Predictive Smart Home Automation

Dear Sir,
I am pleased to submit the technical report for "Helion++: Advanced Predictive Smart Home Automation" as part of the project proposal for the final-year capstone project. The document comprehensively overviews the system's objectives, motivation, scope, and functional requirements.

I hope this document meets your expectations and provides clarity regarding the project's purpose and implementation approach. I have tried my best to fulfill all the requirements and your feedback is welcomed for further improvement.

Yours sincerely,
Dipshikha Das
BSSE-1218
Institute of Information Technology
University of Dhaka



_____

Supervisor Signature

Acknowledgment

I am grateful to the Institute of Information Technology for providing me with the incredible opportunity to work on the project titled “Helion++: Advanced Predictive Smart Home Automation” I am deeply thankful to my supervisor, Dr. Md. Nurul Ahad Tawhid, Associate Professor at the Institute of Information Technology, University of Dhaka, for his invaluable guidance and support in preparing this analysis document. Finally, I wish to acknowledge my classmates for their continuous support and valuable insights, which have been immensely helpful throughout this journey.

Abstract

Helion++ is an enhanced smart home automation framework built upon the original Helion framework, aimed at improving the efficiency and adaptability of automation processes. While the original Helion utilized n-gram-based models to predict automation sequences, its performance was constrained by limited data. Helion++ addresses these limitations by leveraging a large dataset of over 900 routines and execution indicators and employing advanced deep learning techniques such as Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM) networks.

The project introduces a tokenization mechanism to standardize and process user-driven home automation routines, enabling more accurate sequence predictions. Helion++ enhances predictive capabilities by analyzing complex user behavior patterns, facilitating informed scheduling of routines, anomaly detection, and security policy refinement. The integration of deep learning allows Helion++ to continually improve its performance as it learns from increasing datasets, making it a scalable solution for dynamic smart home environments.

By automating routine execution with higher precision and exploring potential integration with smart home ecosystems, Helion++ paves the way for smarter, data-driven automation systems that enhance convenience, security, and efficiency. This project represents a significant step toward creating an adaptive, intelligent framework for next-generation home automation.

Table of Contents

1. Introduction..... 6
1.1 Motivation..... 6
1.2 Scope..... 6
1.3 Purpose of the Document..... 7
2. Description of the Project..... 8
2.1 Quality Function Deployment..... 8
2.1.1 Normal Features..... 8
2.1.2 Expected Features..... 8
2.1.3 Exciting Features..... 8
2.2 Usage Scenario..... 9
2.2.1 Event Prediction..... 9
2.2.2 User Interface for Predictions..... 9
2.2.3 Prediction Flavors..... 9
2.2.4 Model Selection..... 9
3. Scenario-Based Modeling..... 10
3.1 Use Case Diagram..... 10
3.1.1 Helion++..... 10
3.1.2 Modules of Helion++..... 11
3.1.2.1 Modeling and Predictions..... 12
5. Data-based Modeling..... 13
6. Class-based modeling..... 13
6.1 Analysing Classes..... 13
6.2 Class Cards..... 14
6.3 Class Responsibility and Collaboration Diagram..... 16
7. Architectural Design..... 17
7.1 Architectural Context Diagram..... 17
7.2 ArcheTypes..... 18
7.3 Top Level Components..... 20
8. Preliminary Test Plan..... 21
8.1 High-Level Description of Testing Plans..... 21
8.2 Test Cases..... 22
9. Conclusion..... 24

List of Figures

1. Level - 0 of Use Case Diagram..... 10
2. Level - 1 of Use Case Diagram..... 11
3. Level - 1.1 of Use Case Diagram..... 12
4. CRC Diagram..... 17
5. Architectural Diagram..... 18
6. Using MVC for UI..... 18
7. Using layered for Trainer..... 19
8. Using Pipe and Filter for Data Processing..... 19
9.Layered For Predictions..... 20
10.Top Level Components..... 21

1. Introduction

Smart home automation is transforming the way people interact with their living spaces. However, existing systems often rely on basic rule-based mechanisms or manually configured routines, which can be restrictive and inefficient. Helion++ builds upon the original Helion framework, which utilized n-gram models for predicting smart home automation sequences, to create a more adaptive and intelligent solution. By leveraging a larger dataset and incorporating advanced deep learning techniques such as RNNs and LSTMs, Helion++ aims to enhance the prediction and execution of smart home routines.

This document outlines the requirements and scope of the Helion++ project, providing a detailed overview of its goals, features, and implementation approach. The focus is on improving automation precision, analyzing user behavior effectively, and paving the way for integration into smart home ecosystems.

1.1 Motivation

Smart home technology is advancing rapidly, yet many existing systems fail to move beyond rigid, manual setups or basic rule-based algorithms. This limitation results in inefficiencies, unoptimized routines, and a lack of adaptability to dynamic user behaviors.

The original Helion framework showcased potential in predicting smart home automation sequences using n-gram models, but it was constrained by a small dataset. Now, with access to a significantly larger corpus of over 900 routines and execution indicators, there exists a unique opportunity to create a smarter, data-driven automation system.

By tokenizing this larger dataset and applying deep learning models, Helion++ seeks to overcome the limitations of traditional n-gram methods. This enhanced framework will not only improve the precision of automation routines but also provide a robust foundation for anomaly detection, security enhancements, and informed scheduling within smart home environments.

1.2 Scope

Helion++ is designed to revolutionize smart home automation by combining the strengths of data-driven insights with the predictive power of deep learning models. Its scope encompasses:

- **Tokenization and Data Processing:** Efficiently handling a large dataset of smart home routines to identify patterns and user behavior.
- **Enhanced Prediction:** Using advanced deep learning techniques (RNNs and LSTMs) to accurately predict and automate user routines.

- **Practical Integration:** Potentially incorporating the predictive capabilities into an existing smart home platform for real-world applications.
- I**mproved Security:** Supporting anomaly detection and policy specification for safer home automation.

1.3 Purpose of the Document

The purpose of this document is to define the functional and non-functional requirements of the Helion++ project. It serves as a comprehensive guide for the development team, stakeholders, and researchers involved in the project by providing:

- A clear understanding of the system's objectives, features, and design approach.
- A roadmap for implementing enhancements to the original Helion framework using deep learning models.
- A foundation for aligning the project with user expectations and industry standards in smart home automation.
- A means to ensure the project meets its goals of improving automation accuracy, security, and adaptability.

By detailing the requirements, this document lays the groundwork for developing a smarter, more efficient, and scalable home automation system that addresses the shortcomings of existing frameworks.

2. Description of the Project

Helion++ is an advanced smart home automation framework that enhances the original Helion by integrating deep learning techniques like RNN and LSTM. It utilizes a large dataset of over 900 routines and execution indicators to improve the prediction of user-driven automation sequences. By tokenizing events as <device, attribute, action> triplets, Helion++ efficiently processes user behavior patterns, enabling more accurate predictions and informed scheduling.

The framework enhances security, anomaly detection, and routine execution by learning from historical data and adapting to new patterns. With improved scalability and potential integration into smart home ecosystems, Helion++ aims to deliver smarter, more adaptive, and more efficient home automation solutions.

2.1 Quality Function Deployment

Quality Function Deployment (QFD) is a structured methodology used in product development and project management to transform customer needs and expectations into detailed technical specifications and features. It ensures that the final product aligns closely with what customers value the most. The features of helion++ can be categorized as follows:

2.1.1 Normal Features

- **Tokenization of a large dataset:** Ensures data is processed efficiently and forms the foundation of prediction.
- **Event Predictions:** Predict events based on a given sequence.

2.1.2 Expected Features

- **Enhanced predictive capability by analyzing sequences of user behavior:** Improves the system's ability to adapt to user needs by using deep learning.
- **Better Accuracy than the previous model:** Ensures the system performs better than the previous one that was built using the n-gram model.

2.1.3 Exciting Features

- **User-Friendly Interface:** A user-friendly UI will be provided to the users for predicting the event sequences.

2.2 Usage Scenario

Helion++: Smart Home Event Prediction will be designed to predict smart home automation events based on user behavior patterns. It will enable researchers, vendors, or any other user to get predicted scenarios for the smart home system. The usage scenario for the system can be segmented as follows:

2.2.1 Event Prediction

Helion++ uses historical behavioral data to predict future smart home events. The user provides 2-3 sequences of events they expect to occur, and based on these initial inputs, the system generates predictions about the subsequent actions. This helps users anticipate future automation activities within their smart home systems.

2.2.2 User Interface for Predictions

The Helion++ web page provides an intuitive user interface that allows users to input event sequences and view predictions. Users enter the first few sequences of events through the web page, and Helion++ processes this information to generate predictions about what is likely to happen next in the system. The results are displayed clearly on the web page, making it easy for users to understand upcoming automation events.

2.2.3 Prediction Flavors

The web page presents predictions in two distinct flavors:

- Up Flavor (High Likelihood Events): These are events that have a high likelihood of occurring, such as routine behaviors (e.g., "User is likely to dim the lights and turn on the TV at 8:00 PM").
- Down Flavor (Rare Events): These predictions are for less likely or rare occurrences (e.g., "User might turn on the backyard lights at midnight").

2.2.4 Model Selection

The web page allows users to choose between two models to generate predictions:

- RNN (Recurrent Neural Network): Ideal for predicting short-term, sequential behaviors based on the initial event sequences provided by the user.
- LSTM (Long Short-Term Memory): Suitable for predicting more complex, long-term event patterns, capturing deeper insights from user behavior over time.

3. Scenario-Based Modeling

Scenario-based modeling is a software development method that involves creating models to demonstrate how a system responds or behaves in various real-world situations. These scenarios are derived from user interactions, events, or conditions, providing insights into the system's functionality in diverse contexts. By simulating these scenarios, developers can uncover potential issues, make better design decisions, and ensure the software aligns with user needs across different scenarios.

3.1 Use Case Diagram

A use case diagram visually depicts the interaction between a system and its external entities or actors to achieve specific objectives. It illustrates the relationships between use cases, which represent system functionalities, and actors, which represent external users or systems. Use case diagrams are typically created by gathering system requirements, identifying functionalities and interactions, and representing them using software modeling tools.

3.1.1 Helion++

Level - 0:

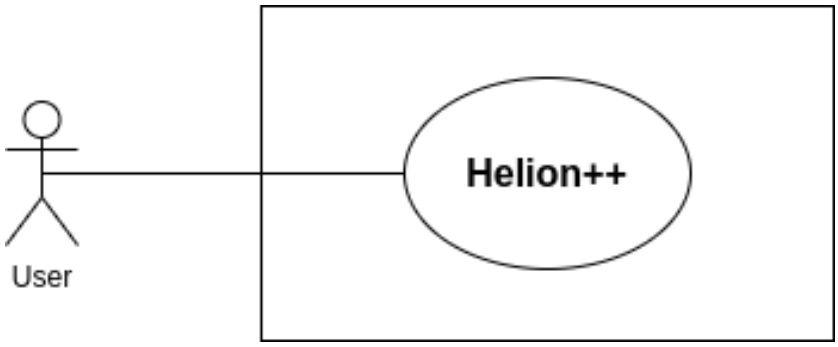


Fig: 1(Level - 0 of Use Case Diagram)

Primary Actor: User
Goal in context: This diagram shows the enhanced home automation prediction system - helion++.

3.1.2 Modules of Helion++

Level: 1

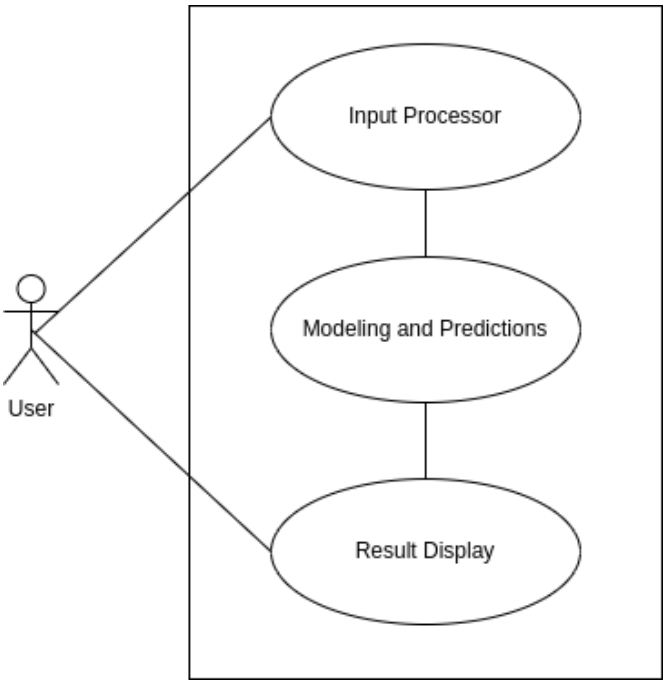


Fig: 2(Level - 1 of Use Case Diagram)

Primary Actor: User
Goal in Context: This diagram shows the modules of Helion++

These modules can be elaborated as follows:

- **Input Processor:** Responsible for processing raw input data, including tokenization, cleaning, and feature extraction. It prepares the data for informed scheduling and predictive modeling.

- **Modeling and Predictions:** Focuses on training and utilizing machine learning models to generate accurate event predictions. It provides both high-likelihood (Up Flavor) and rare (Down Flavor) predictions based on user behavior patterns. The modeling and predictions module is further divided into two components - RNN based modeling and LSTM-based modeling and predictions.

- **Display Result:** Through the provided user-friendly ui the user will be able to see the predicted results on the web page based on their choice of model(RNN or LSTM) and flavor of predictions(Up or Down).

3.1.2.1 Modeling and Predictions

Level: 1.1

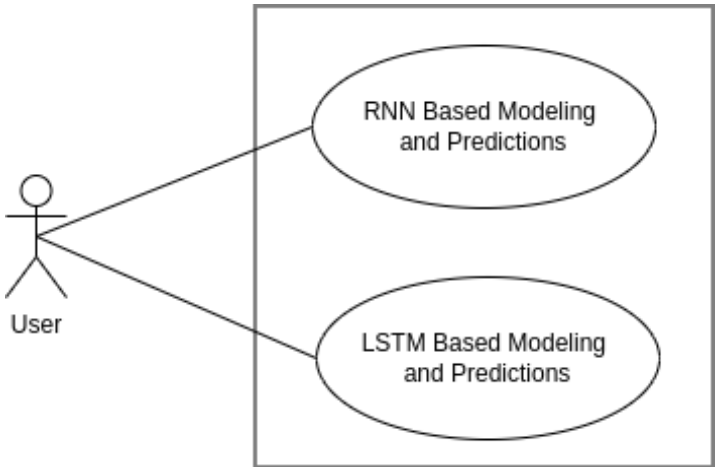


Fig: 3(Level - 1.1 of Use Case Diagram)

Primary Actor: User
Goal in this context: This diagram shows the modules of Modeling and Predictions that are divided into two parts.

The elaboration of these two modules is given below:

- **RNN-Based Modeling and Predictions:** The RNN-based modeling and predictions in Helion++ utilize Recurrent Neural Networks to analyze sequences of user behavior data, allowing the system to predict future home automation events based on temporal patterns. It focuses on short-term event predictions by processing recent user interactions.

- **LSTM-Based Modeling and Predictions:** The LSTM-based modeling and predictions in Helion++ employ Long Short-Term Memory networks to capture long-term dependencies in user behavior. This component improves the system's ability to make more accurate predictions for complex, infrequent events by remembering important past behaviors over extended periods.

5. Data-based Modeling

No database or data storage has been used here. So this part of data-based modeling is irrelevant here.

6. Class-based modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

6.1 Analysing Classes

After identifying the relevant nouns from the scenario, I categorized them into the solution domain by applying General Classification criteria, which include external entities, things, events, roles, organizational units, places, and structures. To refine this list, I evaluated each noun against specific Selection Criteria such as the ability to retain information, the need for services, the presence of multiple or common attributes, shared operations, and essential requirements. Following this thorough analysis, I have identified the following analysis classes

- Model
- ModelTrainer
- ModelPredictor
- Input Processor
- RNN Based Modeling
- LSTM Based Modeling
- UI
- Helion++

6.2 Class Cards

Class cards for Helion++ are given below

Helion++	
Attributes	**Methods**
- trainer - predictor - configurations	+ initialize() + train() + predict() +getResult()
Responsibility	**Collaborator**
● Runs the entire system ● Access a mediator ● Takes input and provides output	● ModelTrainer ● ModelPredictor
Model	
Attributes	**Methods**
- modelTypes - parameters - trainedModel - accuracy	+ loadData() + evaluate() + saveModel() + loadModel() + getParsedData()
Responsibility	**Collaborator**
● Configure the weights ● Run predictions	
Input Processor	
Attributes	**Methods**
- dataFormat - rawData - parsedData	+ getData() + parse() + generateTimeSeries() + validate() + export()
Responsibility	**Collaborator**
● Parse/preprocess data, ● Handle informed scheduling.	

ModelTrainer	
Attribute	**Methods**
- model - input parser	+ parseData() + train()
Responsibility	**Collaborator**
• Trains Model	• Input Processor • Model
ModelPredictor	
Attribute	**Method**
- model - input parser	+ parseInput() + predict()
Responsibility	**Collaborator**
• Gives Predictions	• Model • Input Processor
RNN Based Modeling	
Attributes	**Methods**
- rnnLayers - hiddenUnits - activationFunction	+ train() + predict() + tuneHyperParameters()
Responsibility	**Collaborator**
• RNN training • Short-term predictions.	
LSTM Based Modeling	
Attributes	**Methods**
- lstmLayers - cellStateSize - dropOutRate	+ train() + predict() + tuneHyperParameters()
Responsibility	**Collaborator**
• LSTM training	

• Long-term predictions.	

6.3 Class Responsibility and Collaboration Diagram

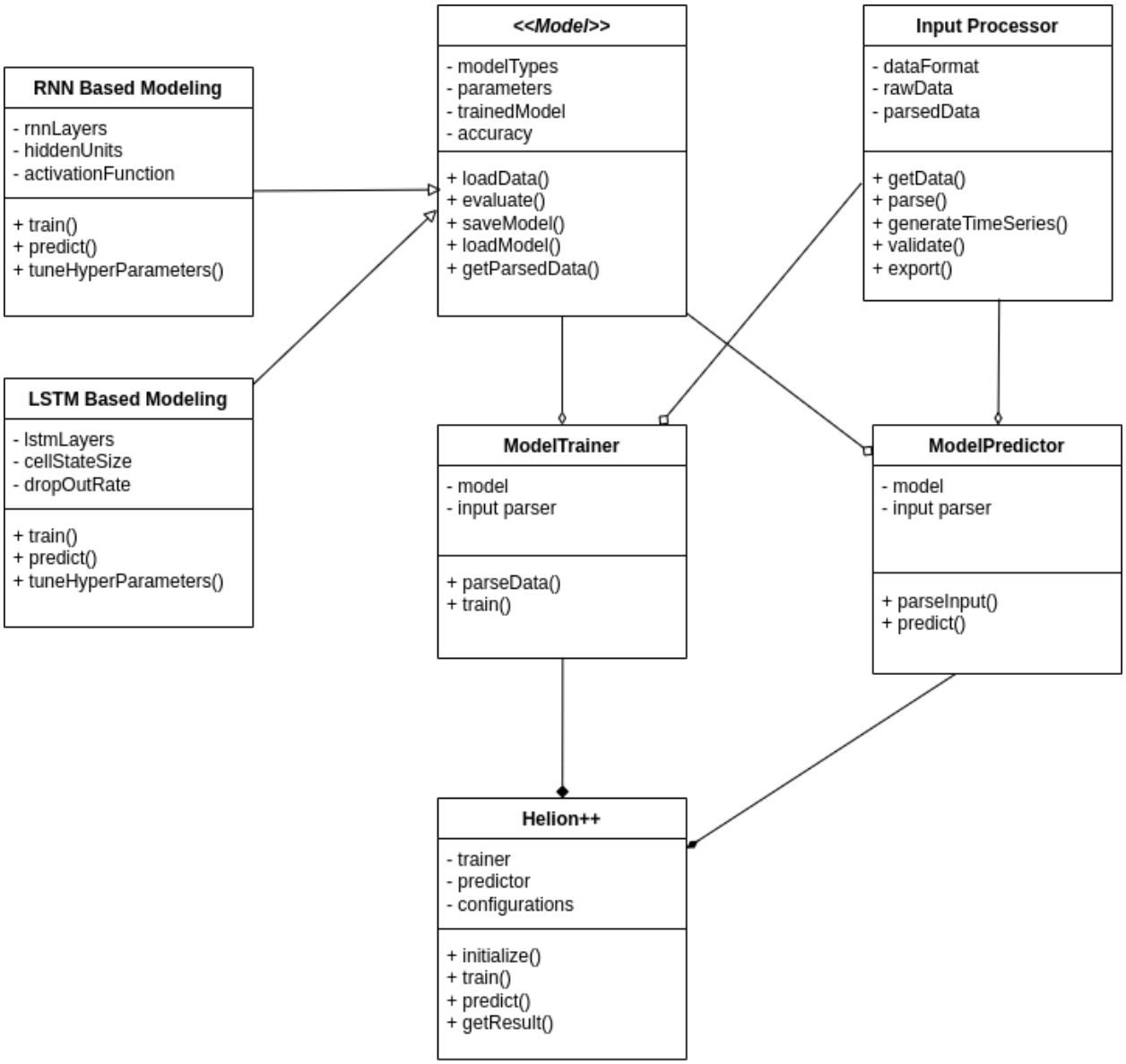


Fig: 4(CRC Diagram)

7. Architectural Design

Architectural design is a high-level blueprint in software engineering that visually represents the structure and organization of a software system. It highlights the key components or modules, their interactions, and the overall system architecture, providing a clear framework for implementation.

7.1 Architectural Context Diagram

The architectural context diagram for Helion++ is given below:

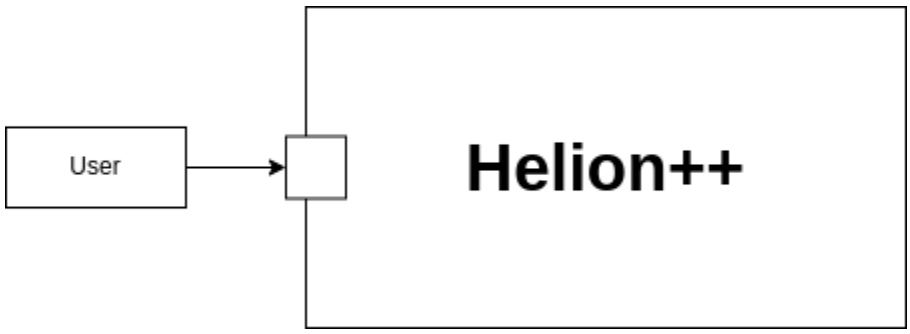


Fig: 5(Architectural Diagram)

7.2 ArcheTypes

The archetypes for the tool are defined below:

MVC for UI

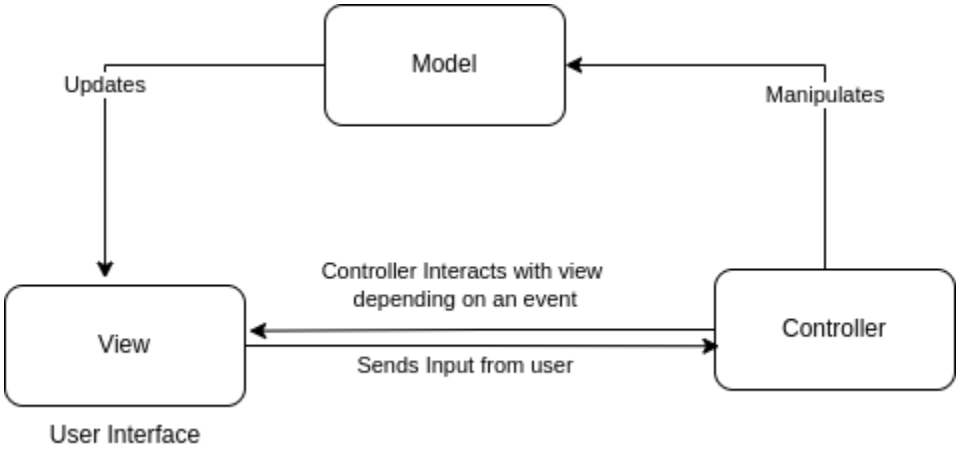


Fig: 6(MVC for UI)

Layered Pattern For Trainer

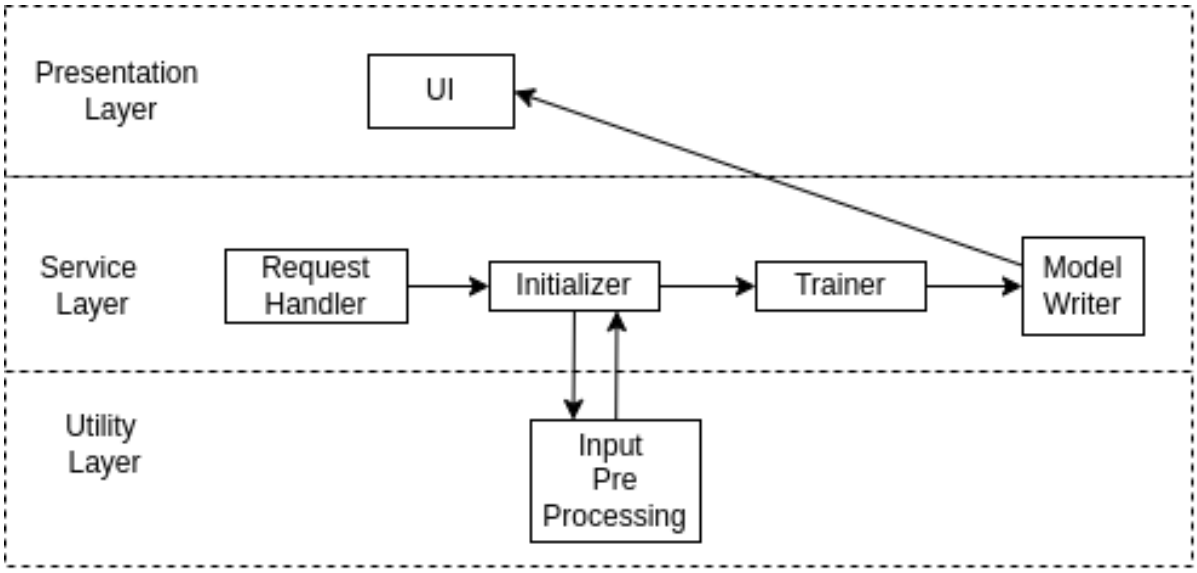


Fig: 7(Layered for Trainer)

Further Pipe and Filter Can be Used for Input Preprocessing

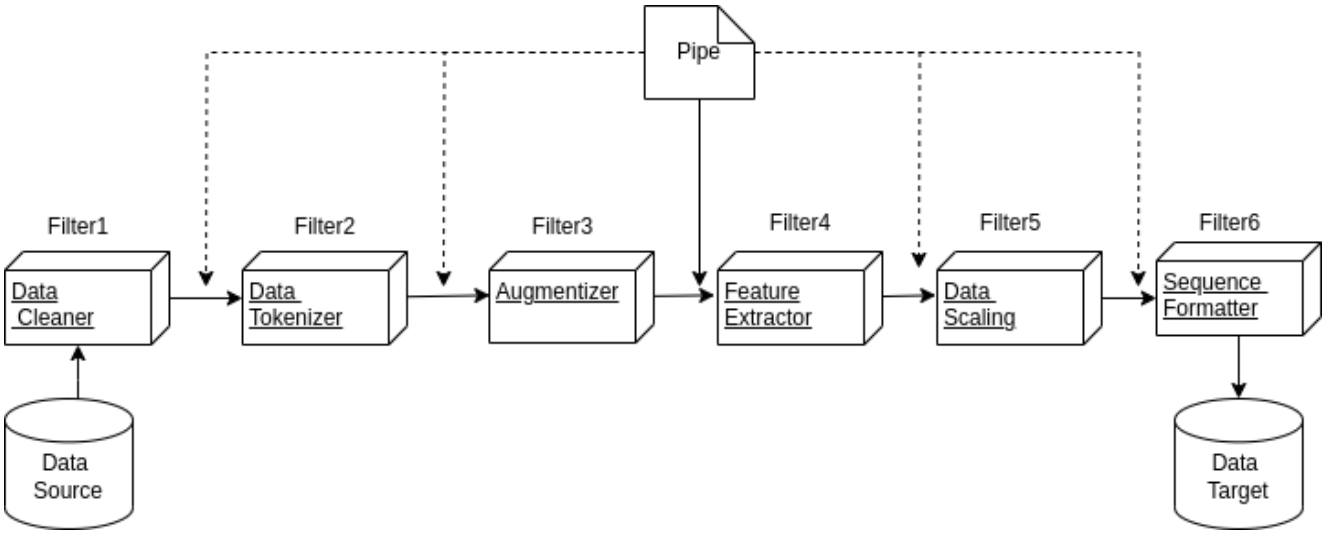


Fig: 8(Pipe and Filter for Data Processing)

Layered for Predictions

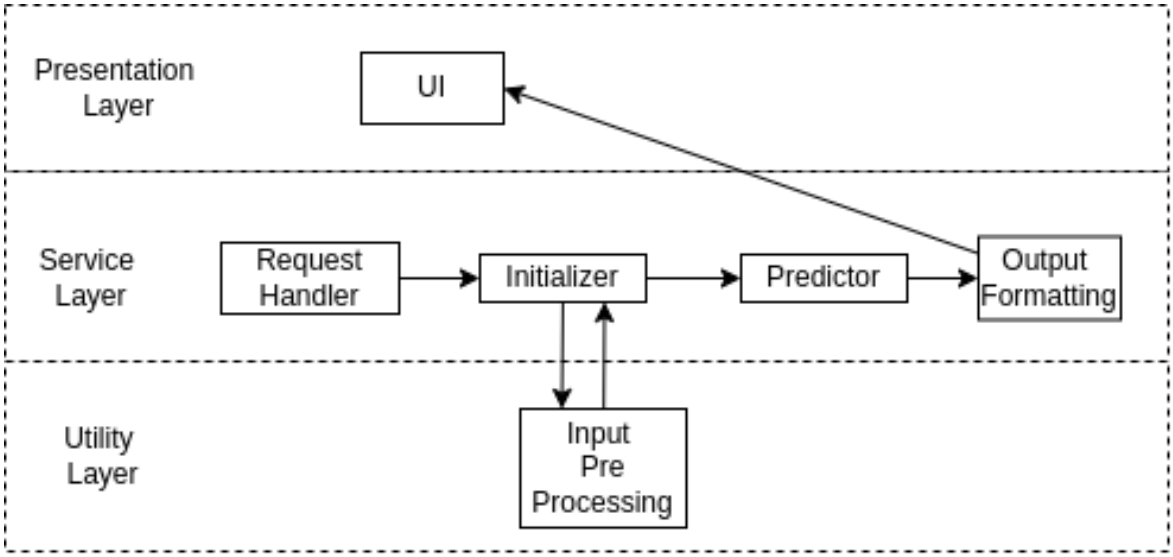


Fig: 9(Layered Predictions)

7.3 Top Level Components

The top-level components of the architecture are given below:

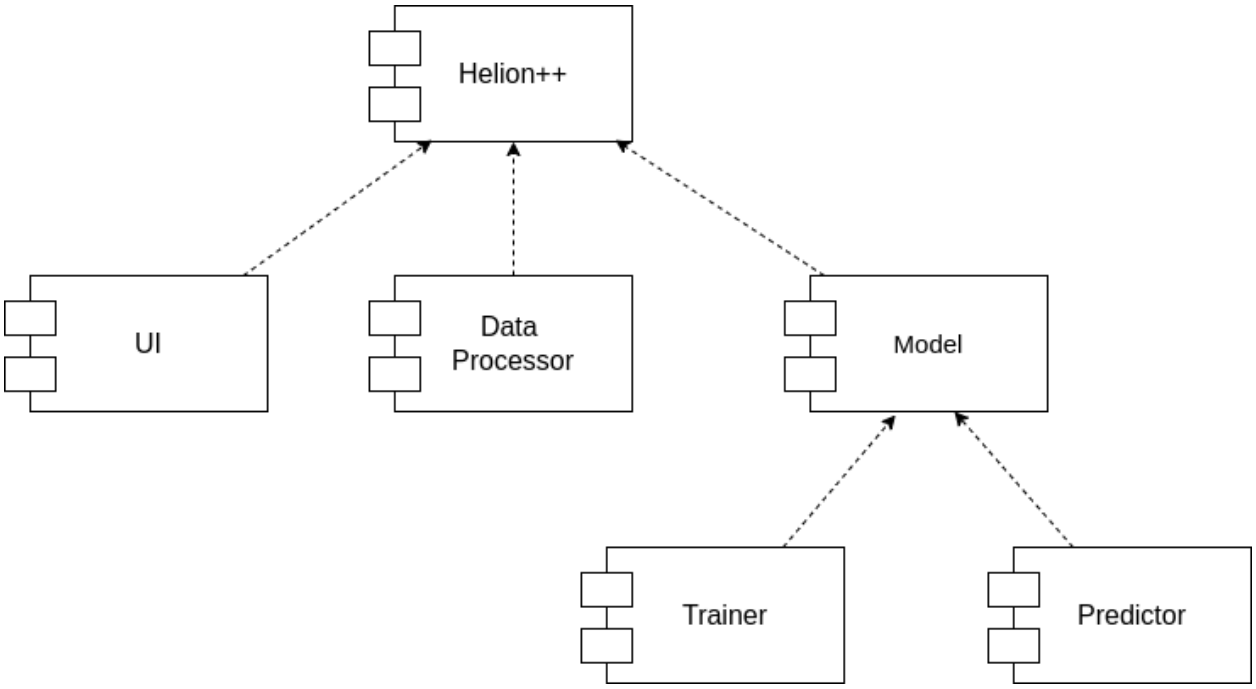


Fig: 10(Top Level Components)

8. Preliminary Test Plan

8.1 High-Level Description of Testing Plans

The test cases for Helion++ are designed to:

- **Validate Functionality:** Ensure components like the input processor, trainer, and predictor work as expected.
- **Ensure Reliability:** Confirm accurate predictions and robust handling of diverse user inputs.
- **Enhance User Experience:** Provide user-friendly, customizable, and relevant predictions.
- **Test Performance:** Check scalability and responsiveness with large datasets and integrations.
- **Identify Bugs:** Detect and address errors or edge cases.
- **Support Improvements:** Lay a foundation for testing future updates and features.

The goal is to deliver a reliable, accurate, and user-centric smart home prediction API.

8.2 Test Cases

Test cases designed for this project are described below:

- **Test Case 1:** Web Page Accessibility and Activation
    - Test Scenario:
        - Access the Helion++ web page via a browser.
        - Ensure that the web page loads properly and is ready for interaction.
    - Expected Outcome:
        - The web page loads without errors.
        - The page displays a clean, fully functional UI, and all components (buttons, input fields, etc.) are accessible.
        - No broken links, missing resources, or loading issues.

- **Test Case 2:** Data Parsing
    - Test Scenario:
        - Input a dataset containing trigger action events and execution indicators.
        - Let Helion++'s input processor the input.
    - Expected Outcome:
        - The input processor tokenizes, cleans, and extracts features correctly from the input data.

        - The processed data is structured in a format usable by the trainer and predictor components.

- **Test Case 3:** Routine Scheduling (Informed Scheduling)
    - Test Scenario:
        - Input routines with a time range, day range, and frequency indicators.
        - Let the input processor generate a month-long time series of routines.
    - Expected Outcome:
        - Routines are correctly placed in appropriate time slots based on execution indicators.
        - Uncertain routines are randomly distributed across remaining time slots.
        - The output is an ordered execution sequence for the month.

- **Test Case 4:** Basic Event Prediction
    - Test Scenario:
        - Provide a sequence of 2–3 events as input.
        - Request predictions for the next likely event(s).
    - Expected Outcome:
        - The web page generates predictions for future events accurately.
        - Predictions include both Up Flavor (high likelihood events) and Down Flavor (rare events).

- **Test Case 5:** User-Centric Filtering
    - Test Scenario:
        - Input a request for predictions filtered by Up Flavor (high likelihood) and Down Flavor (rare events).
    - Expected Outcome:
        - The API filters predictions based on the user's preferences.
        - Predictions for Up and Down Flavors are clearly categorized.

- **Test Case 6:** Performance and Scalability
    - Test Scenario:
        - Input a large dataset containing historical data for multiple devices and routines.
        - Request predictions from the web page.
    - Expected Outcome:
        - The API processes the data and generates predictions within a reasonable time frame.
        - System performance remains stable with no significant latency or errors.

- **Test Case 7:** Web Page and Backend Integration
    - Test Scenario:
        - Interact with the web page by selecting event sequences, prediction models, and prediction flavor preferences.
        - Ensure that the UI communicates correctly with the backend to fetch and display predictions.
    - Expected Outcome:
        - The web page correctly sends user inputs to the backend (via API calls or other mechanisms).
        - The backend generates predictions based on the selected inputs (model and flavor) and sends results back to the web page.
        - Predictions are displayed accurately, and any status updates (loading, errors) are shown to the user.

- **Test Case 8:** Usability Testing
        - Test Scenario:
        - Test the overall user experience of the web page, ensuring that it's intuitive and easy to navigate.
        - Users should be able to input data, select models, and view predictions without confusion.
    - Expected Outcome:
        - The web page is user-friendly, with clear instructions and an intuitive layout.
        - Users can easily input their event sequences and select prediction options.
        - Results are displayed in an easy-to-understand format, with all relevant information (predictions, model used, flavor) clearly shown.

- **Test Case 9:** Cross-Browser Compatibility
    - Test Scenario:
        - Test the Helion++ web page on multiple browsers (e.g., Chrome, Firefox, Edge, Safari) to ensure it is fully functional across them.
    - Expected Outcome:
        - The web page loads and functions properly on all tested browsers.
        - No layout issues, missing content, or broken functionality when accessed through different browsers.

9. Conclusion

The documentation for Helion++ has been created to provide a clear and structured outline of the project's objectives, features, and functionalities. It serves as a foundational guide for developers, stakeholders, and users, ensuring a shared understanding of the system's requirements and design. By detailing use cases, class components, architectural components, and expected outcomes, this document lays the groundwork for consistent development, effective testing, and seamless integration into smart home systems. Ultimately, it ensures that Helion++ aligns with user needs while maintaining scalability and reliability for future enhancements. I have tried to clearly state the workflow of the total system. I hope this report can help the reader have an appropriate understanding of the technical details of this tool.

Reference

[1] Manandhar, Sunil, et al. ‘Towards a Natural Perspective of Smart Homes for Practical Security and Safety Analyses’. 2020 IEEE Symposium on Security and Privacy (SP), 2020, pp. 482–499, https://doi.org/10.1109/SP40000.2020.00062.

[2] Mandal, Prianka, et al. ‘Helion: Enabling Natural Testing of Smart Homes’. Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, vol. 5, ACM, 2023, pp. 2147–2151, https://doi.org/10.1145/3611643.3613095. ESEC/FSE ’23.